

Lab Report of Optimization Techniques

Submitted By

Sourabh Jain

(Reg. No. 22BCE8428)



Submitted To

Department of Mathematics

School of Advanced Sciences (SAS)

VIT-AP, University, Andhra Pradesh

Lab Instructor: Prof. Komandla Mahipal Reddy

MAT2003 - Optimization Techniques | **L51+L52** | 301-CB |

AP2024254000080

Contents:

Sl. No.	Name of Experiment	Page
1.	<u>Linprog (f,A,b)</u>	1-3
2.	<u>Simplex Method</u>	3-8
3.	<u>Big M Method</u>	9-16
4.	<u>Duality (Primal to Dual)</u>	17-25
5.	<u>Dual- Simplex Method</u>	26-30
6.	<u>Transportation Problem (North West Corner Method)</u>	31-36
7.	<u>Assignment Problem</u>	36-42
8.	<u>Region Elimination Method (Fibonacci Search)</u>	43-44
9.	<u>Newton - Raphson Method</u>	45-46

DAY-1

Linprog (f,A,b)

Date: 17 Jan 2025

Code

```
clc
clear
```

```
% Inputs : f,A,b,lb
```

```
f = [-7; -5]; % Negated for maximization
```

```
% Coefficients of the inequality constraints
```

```
A = [4 3;
     2 1];
```

```
% Right-hand side of the inequality constraints
```

```
b = [240;100];
```

```
% Bounds for x1 and x2 (non-negative constraints)
```

```
lb = [0; 0]; % x, y >= 0
```

```
% Use linprog to solve the linear program
```

```
% Syntax: linprog(f, A, b, Aeq, beq, lb, ub)
```

```
[x, fval, exitflag, output] = linprog(f, A, b, [], [], lb, []);
```

```
% Display the results
```

```
if exitflag == 1
```

```
    disp('Optimal solution found:');
```

```
    disp('Values of x and y:');
```

```
    disp(x);
```

```
    disp('Optimal value of Z:');
```

```
    disp(-fval); % Negate the result to get the actual maximum value
```

```
else
```

```
    disp('No optimal solution found.');
```

```
end
```

Question 1 : Furniture company: A furniture company produces tables and chairs. The production process for each is similar in that both require a certain number of hours of carpentry work and certain number of labour hours in the painting department. Each table takes 4 hours of carpentry and 2 hours in the painting department. Each chair requires 3 hours of carpentry and 1 hour in the painting department. During the current production period, 240 hours of carpentry time are available and 100 hours in painting is available. Each table sold yields a profit of E7, each chair is sold for a E5 profit. (where E5 means 5 euros and E7 means 7 euros)

Input -> f=[-7 -5] A=[4 3; 2 1] b=[240;100]

Output ->

Optimal solution found.

Optimal solution found:

Values of x and y:

30

40

Optimal value of Z:

410

Question 2 : A patient in a hospital is required to have at least 84 units of drug A and 120 units of drug B each day. Each gram of substance M contains 10 units of drug A and 8 units of B, and each gram of substance N contains 2 units of drug A and 4 units of drug B. Now suppose that both M and N contain an undesirable drug C, 3 units per gram in M and 1 unit per gram in N. How many grams of substance M and N should be mixed to meet the minimum daily requirement at the same time minimize the intake of drug C? How many units of the desirable drug C will be in this mixture?

Input -> $f=[3 \ 1]$ $A=[-10 \ -2; -8 \ -4]$ $b=[-84;-120]$

Output ->

Optimal solution found.

Optimal solution found:

Grams of substance M:

4

Grams of substance N:

22

Minimum intake of drug C:

34

Question 3 : A farmer has a cropland of 120 acres. The farmer claims that the most critical limitations on production are land acreage, amount of fertilizer, and labour. The amounts of labour hours and fertilizer available to the farmer are 700 hours and 200 pounds (lb), respectively. Available data indicate that 100 bushels of corn require 7 hours of labour, 2 acres of land, and 10 lb of fertilizer. In addition, the data reveal that 100 bushels of soybeans require 30 hours of labour, 4 lb of fertilizer, and 1 acre of land. Moreover, the prices of corn and soybeans are determined by market forces at 3.00 for corn and 5.00 for soybeans. But each bushel of corn and soybeans costs the farmer 2.80 and 4.55, respectively. How can our farmer allocate economic resources efficiently in order to maximize profit?

Input -> $f = [-0.20; -0.45]$ $A = [2 \ 1; 7 \ 30; 10 \ 4]$ $b = [120; 700; 200]$

Output ->

Optimal solution found.

Optimal solution found:
Bushels of Corn (in 100s):
200/17

Bushels of Soybeans (in 100s):
350/17

Maximum Profit:
395/34

Simplex Method

Date: 17 Jan 2025

Code

```
clear
clc

% Inputs : A,b,C
% Objective function Z = 10x + 12y ( Maximize )
A = [4 6 1 0; 3 8 0 1]; % Coefficients of constraints
b = [240; 300]; % RHS of constraints
C = [10 12 0 0]; % Coefficients of the objective function
Cb = [0; 0]; % Initial basic variable coefficients
% Z=10x + 12y + 0S1 + 0S2 ( Maximize )
[m, n] = size(A); % Dimensions of A
Zj = zeros(1, n); % To store Zj - Cj values

% Augmented simplex table
S = [A, b];

% Simplex Iteration
while true
    % Step 1: Calculate Zj - Cj
    for i = 1:n
        Zj(i) = Cb' * A(:, i) - C(i);
    end

    % Display Zj-Cj
    disp("Zj-Cj:");
    disp(Zj);

    % Step 2: Check optimality
    if all(Zj >= 0)
        disp("Optimal solution found.");
        break;
    else
```

```

    disp("Solution is not optimal, proceeding with iteration.");
end

% Step 3: Find pivot column (most negative Zj-Cj)
[min_Zj, pivot_col] = min(Zj);
disp("Pivot Column: " + pivot_col);

% Step 4: Find pivot row (minimum positive ratio)
ratios = b ./ A(:, pivot_col);
ratios(ratios <= 0) = Inf; % Ignore negative or zero ratios
[min_ratio, pivot_row] = min(ratios);

if isinf(min_ratio)
    error("The problem is unbounded.");
end
disp("Pivot Row: " + pivot_row);

% Step 5: Perform pivoting
key_element = A(pivot_row, pivot_col);
disp("Key Element: " + key_element);

% Normalize the pivot row
S(pivot_row, :) = S(pivot_row, :) / key_element;

% Update the other rows
for i = 1:m
    if i ~= pivot_row
        S(i, :) = S(i, :) - S(i, pivot_col) * S(pivot_row, :);
    end
end

% Update Cb and b
Cb(pivot_row) = C(pivot_col);
b = S(:, end);

% Update A
A = S(:, 1:n);

disp("Modified Simplex Table:");
disp(S);
end

% Step 6: Display the final solution
disp("Final Simplex Tableau:");
disp(S);

% Initialize basic variables solution as zeros
basic_solution = zeros(n, 1);

% Loop through each row of A

```

```

for i = 1:m
    % Find the column with a single '1' in the current row
    for j = 1:n
        if A(i, j) == 1
            basic_solution(j) = b(i); % Assign corresponding value from b
            break; % Exit the loop once the column is found
        end
    end
end

% Display results
disp("Optimal Solution:");
disp(basic_solution);

% Calculate and display the optimal value
disp("Optimal Value:");
optimal_value = 0;
for i = 1:m
    optimal_value = optimal_value + Cb(i) * b(i);
end
disp(optimal_value);

```

Q1. Resource Allocation in Industry: A firm that produces two products has limitations on its machine and labour time. There are 240 hours of machine time and 300 hours of labour time available during the production period. A unit of the first product requires 4 hours of machine time and 3 hours of labour time. The second product takes 6 and 8 hours of machine and labour time, respectively. A unit of product 1 and product 2 generates profits of 10.00 and 12.00, respectively. The economic problem is to find the levels of products 1 and 2 that maximize profit and allocate economic resources efficiently.

Input -> $A = \begin{bmatrix} 4 & 6 & 1 & 0 \\ 3 & 8 & 0 & 1 \end{bmatrix}$; $b = \begin{bmatrix} 240 \\ 300 \end{bmatrix}$; $C = \begin{bmatrix} 10 & 12 & 0 & 0 \end{bmatrix}$;

Output ->

Zj-Cj:

-10 -12 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 2

Key Element: 8

Modified Simplex Table:

1.7500	0	1.0000	-0.7500	15.0000
0.3750	1.0000	0	0.1250	37.5000

Zj-Cj:

-5.5000 0 0 1.5000

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 1

Key Element: 1.75

Modified Simplex Table:

1.0000	0	0.5714	-0.4286	8.5714
0	1.0000	-0.2143	0.2857	34.2857

Zj-Cj:

0 0 3.1429 -0.8571

Solution is not optimal, proceeding with iteration.

Pivot Column: 4

Pivot Row: 2

Key Element: 0.28571

Modified Simplex Table:

1.0000	1.5000	0.2500	0	60.0000
0	3.5000	-0.7500	1.0000	120.0000

Zj-Cj:

0 3.0000 2.5000 0

Optimal solution found.

Final Simplex Tableau:

1.0000	1.5000	0.2500	0	60.0000
0	3.5000	-0.7500	1.0000	120.0000

Optimal Solution:

60.0000

0

0

120.0000

Optimal Value:

600.0000

Q2. Manufacturing products: A manufacturer produces two products A and B. Both the products are processed on two different machines. The available capacity of first machine is 12 hours and that of second machine is 9 hours per day. Each unit of product A requires 3 hours on both machines and each unit of product B requires 2 hours on first machine and 1 hour on second machine. Each unit of product A is sold at Rs 7 profit and B at a profit of Rs 4. Find the production level per day for maximum profit.

Input -> $A = \begin{bmatrix} 3 & 2 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{bmatrix}$ $b = [12; 9]$ $C = [7 \ 4 \ 0 \ 0]$;

Output ->

Zj-Cj:

-7 -4 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 2

Key Element: 3

Modified Simplex Table:

0	1.0000	1.0000	-1.0000	3.0000
1.0000	0.3333	0	0.3333	3.0000

Zj-Cj:

0 -1.6667 0 2.3333

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 1

Key Element: 1

Modified Simplex Table:

0	1.0000	1.0000	-1.0000	3.0000
1.0000	0	-0.3333	0.6667	2.0000

Zj-Cj:

0 0 1.6667 0.6667

Optimal solution found.

Final Simplex Tableau:

0	1.0000	1.0000	-1.0000	3.0000
1.0000	0	-0.3333	0.6667	2.0000

Optimal Solution:

2
3
0
0

Optimal Value:

26

Q3. Maximum Selling in each city: A businessman can travel to city A, city B, or city C. It is 122 miles to city A, 237 miles to city B, and 307 miles to city C. He can travel up to 3000 miles. Dining and other expenses are \$95 in city A, \$130 in city B, and \$180 in city C. Her

expense account allows her to spend \$2000. A trip to city A will generate \$800 in sales, while a trip to city B will generate \$1300 and a trip to city C will generate \$1800. How many trips should he make to each city to maximize sales? Write the initial simplex tableau.

Input - $\rightarrow A = [122 \ 237 \ 307 \ 1 \ 0 \ ; \ 95 \ 130 \ 180 \ 0 \ 1] \ b = [3000; \ 2000] \ C = [800 \ 1300 \ 1800 \ 0 \ 0];$

Output $\rightarrow$

Zj-Cj:

-800 -1300 -1800 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 3

Pivot Row: 1

Key Element: 307

Modified Simplex Table:

0.3974 0.7720 1.0000 0.0033 0 9.7720
23.4691 -8.9577 0 -0.5863 1.0000 241.0423

Zj-Cj:

-84.6906 89.5765 0 5.8632 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 2

Key Element: 23.4691

Modified Simplex Table:

0 0.9237 1.0000 0.0132 -0.0169 5.6905
1.0000 -0.3817 0 -0.0250 0.0426 10.2706

Zj-Cj:

0 57.2519 0 3.7474 3.6086

Optimal solution found.

Final Simplex Tableau:

0 0.9237 1.0000 0.0132 -0.0169 5.6905
1.0000 -0.3817 0 -0.0250 0.0426 10.2706

Optimal Solution:

10.2706

0

5.6905

0

0

Optimal Value:

1.8459e+04

DAY-2

Big M Method

Date: 24 Jan 2025

Code

```
clear
clc

% Inputs : A b C Cb
% Problem: Food P and Q (Big M Method)
% (Z = 60x + 80y + 0S1 + 0S2 - M*A1 - M*A2)
% Constraints:
% 3x + 4y >= 8
% 5x + 2y >= 11
% x,y,S1,S2,A1,A2 >= 0
% Standard forms:
% 3x + 4y - S1 + A1 = 8
% 5x + 2y - S2 + A2 = 11

% Big M method parameters
M = 10000; % Large penalty value

% Coefficient matrix for constraints
A = [3 4 -1 0 1 0; % Row for constraint 1
     5 2 0 -1 0 1]; % Row for constraint 2

% RHS of constraints
b = [8; 11];

% Objective function coefficients (Z = 60x + 80y + 0S1 + 0S2 - M*A1 - M*A2)
C = [-60 -80 0 0 -M -M]; % Coefficients of the objective function
Cb = [-M; -M]; % Initial basic variable coefficients (artificial variables)

% Dimensions of A
[m, n] = size(A);

% Initialize Zj-Cj values
Zj = zeros(1, n);

% Augmented simplex table
S = [A, b];

% Simplex Iteration
while true
    % Step 1: Calculate Zj - Cj
    for i = 1:n
        Zj(i) = Cb' * A(:, i) - C(i);
    end
end
```

```

% Display Zj-Cj
disp("Zj-Cj:");
disp(Zj);

% Step 2: Check optimality
if all(Zj >= 0)
    disp("Optimal solution found.");
    break;
else
    disp("Solution is not optimal, proceeding with iteration.");
end

% Step 3: Find pivot column (most negative Zj-Cj)
[min_Zj, pivot_col] = min(Zj);
disp("Pivot Column: " + pivot_col);

% Step 4: Find pivot row (minimum positive ratio)
ratios = b ./ A(:, pivot_col);
ratios(ratios <= 0) = Inf; % Ignore negative or zero entries
[min_ratio, pivot_row] = min(ratios);

% Check for unboundedness
if isinf(min_ratio)
    error("The problem is unbounded.");
end
disp("Pivot Row: " + pivot_row);

% Step 5: Perform pivoting
key_element = A(pivot_row, pivot_col);
disp("Key Element: " + key_element);

% Normalize the pivot row
S(pivot_row, :) = S(pivot_row, :) / key_element;

% Update the other rows
for i = 1:m
    if i ~= pivot_row
        S(i, :) = S(i, :) - S(i, pivot_col) * S(pivot_row, :);
    end
end

% Update Cb and b
Cb(pivot_row) = C(pivot_col);
b = S(:, end);

% Update A
A = S(:, 1:n);

% Display modified simplex table
disp("Modified Simplex Table:");

```

```

    disp(S);
end

% Step 6: Display the final solution
disp("Final Simplex Tableau:");
disp(S);

% Initialize basic variable solutions as zeros
basic_solution = zeros(n, 1);

% Extract basic variables from tableau
for i = 1:m
    % Find the column with a single '1' in the current row
    for j = 1:n
        if A(i, j) == 1 && sum(A(:, j) == 1) == 1
            basic_solution(j) = b(i); % Assign value from RHS
            break;
        end
    end
end
end

% Display results
disp("Optimal Solution:");
disp(basic_solution);

% Calculate and display the optimal value
disp("Optimal Value:");
optimal_value = Cb' * b;
disp(-1 * optimal_value); % Negate for minimization problem

```

Q1. Sudhama wishes to mix two types of food P and Q in such way that the vitamin contents of the mixture contain atleast 8 units of vitamin A and 11 units of vitamin B. Food P costs Rs 60/kg and Food Q costs Rs. 80/kg. Food P contains 3 units/kg of vitamin A and 5 units/kg of Vitamin B while food Q contains 4 units/kg of vitamin A and 2 units/kg of vitamin B. Determine the minimum cost of mixture.

Input -> $A = \begin{bmatrix} 3 & 4 & -1 & 0 & 1 & 0 \\ 5 & 2 & 0 & -1 & 0 & 1 \end{bmatrix}$ $b = [8; 11]$ $C = [-60 \ -80 \ 0 \ 0 \ -M \ -M]$; $C_b = [-M; -M]$;

Output ->

Zj-Cj:

-79940 -59920 10000 10000 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 2

Key Element: 5

Modified Simplex Table:

0	2.8000	-1.0000	0.6000	1.0000	-0.6000	1.4000
1.0000	0.4000	0	-0.2000	0	0.2000	2.2000

Zj-Cj:

1.0e+04 *

0	-2.7944	1.0000	-0.5988	0	1.5988
---	---------	--------	---------	---	--------

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 1

Key Element: 2.8

Modified Simplex Table:

0	1.0000	-0.3571	0.2143	0.3571	-0.2143	0.5000
1.0000	0	0.1429	-0.2857	-0.1429	0.2857	2.0000

Zj-Cj:

0	0	20	0	9980	10000
---	---	----	---	------	-------

Optimal solution found.

Final Simplex Tableau:

0	1.0000	-0.3571	0.2143	0.3571	-0.2143	0.5000
1.0000	0	0.1429	-0.2857	-0.1429	0.2857	2.0000

Optimal Solution:

2.0000

0.5000

0

0

0

0

Optimal Value:

160

Q2. An animal feed company must produce 200 kg of a mixture consisting of ingredients X1, and X2 daily. X1 cost Rs. 3 per kg and X2 Rs. 8 per kg. Not more than 80 kg of X1 can be used, and at least 60 kg of X2 must be used. Find how much of each ingredient should be used if the company wants to minimise cost.

Input -> $A = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 1 \end{bmatrix}$; $b = [200; 80; 60]$; $C = [-3 \ -8 \ 0 \ 0 \ -M \ -M]$;

$C_b = [-M; 0; -M]$;

Output ->

Zj-Cj:

-999997 -1999992 0 1000000 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 3

Key Element: 1

Modified Simplex Table:

1	0	0	1	1	-1	140
1	0	1	0	0	0	80
0	1	0	-1	0	1	60

Zj-Cj:

-999997 0 0 -999992 0 1999992

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 2

Key Element: 1

Modified Simplex Table:

0	0	-1	1	1	-1	60
1	0	1	0	0	0	80
0	1	0	-1	0	1	60

Zj-Cj:

0 0 999997 -999992 0 1999992

Solution is not optimal, proceeding with iteration.

Pivot Column: 4

Pivot Row: 1

Key Element: 1

Modified Simplex Table:

0	0	-1	1	1	-1	60
1	0	1	0	0	0	80
0	1	-1	0	1	0	120

Zj-Cj:

0	0	5	0	999992	1000000
---	---	---	---	--------	---------

Optimal solution found.

Final Simplex Tableau:

0	0	-1	1	1	-1	60
1	0	1	0	0	0	80
0	1	-1	0	1	0	120

Optimal Solution:

80

120

0

60

0

0

Optimal Value:

1200

Q3 The Subala company during the festival season combines two factors A and B to form a gift pack which must weigh 5 kg. At least 2 kg. of A and not more than 4 kg. of B should be used. The net profit contribution to the company is Rs. 5 per kg. for A and Rs. 6 per kg. for B. Formulate LP Model to find the optimal factor mix.

**Input -> $A = [1 \ 1 \ 0 \ 0 \ 1 \ 0; \ 1 \ 0 \ -1 \ 0 \ 0 \ 1; \ 0 \ 1 \ 0 \ 1 \ 0 \ 0]$; $b = [5; 2; 4]$; $C = [5 \ 6 \ 0 \ 0 \ -M \ -M]$;
 $C_b = [-M; -M; 0]$;**

Output ->

Zj-Cj:

-20005 -10006 10000 0 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 2

Key Element: 1

Modified Simplex Table:

0	1	1	0	1	-1	3
1	0	-1	0	0	1	2
0	1	0	1	0	0	4

Zj-Cj:

0 -10006 -10005 0 0 20005

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 1

Key Element: 1

Modified Simplex Table:

0	1	1	0	1	-1	3
1	0	-1	0	0	1	2
0	0	-1	1	-1	1	1

Zj-Cj:

0 0 1 0 10006 9999

Optimal solution found.

Final Simplex Tableau:

0	1	1	0	1	-1	3
1	0	-1	0	0	1	2
0	0	-1	1	-1	1	1

Optimal Solution:

2

3

0

1

0

0

Optimal Value:

28

DAY-3

Duality (Primal to Dual)

Date: 14 Feb 2025

Code

```
clear
clc

% Inputs : A b C

% Primal : Drug A >=84
%Drug B >=120
% M N
%A 10 2 >=84
%B 8 4 >=120
%C = 3x + y ( Minimize )

% Dual:
% Constraints:
% 10x+8y<=3
% 2x+4y<=1
% Z= 84x+120y
% x,y,S1,S2>=0
% Standard forms:
% 10x+8y+S1=3
% 2x+4y+S2=1

% Coefficient matrix for constraints
A = [10 8 1 0;
     2 4 0 1 ];

% RHS of constraints
b = [3;1];

% Objective function coefficients (Z=84x+120y+0S1+0S2)
C =[84 120 0 0]; % Coefficients of the objective function
Cb = [0;0]; % Initial basic variable coefficients

% Dimensions of A
[m, n] = size(A);

% Initialize Zj-Cj values
Zj = zeros(1, n);
```

```

% Augmented simplex table
S = [A, b];

% Simplex Iteration
while true
    % Step 1: Calculate Zj - Cj
    for i = 1:n
        Zj(i) = Cb' * A(:, i) - C(i);
    end

    % Display Zj-Cj
    disp("Zj-Cj:");
    disp(Zj);

    % Step 2: Check optimality
    if all(Zj >= 0)
        disp("Optimal solution found.");
        break;
    else
        disp("Solution is not optimal, proceeding with iteration.");
    end

    % Step 3: Find pivot column (most negative Zj-Cj)
    [min_Zj, pivot_col] = min(Zj);
    disp("Pivot Column: " + pivot_col);

    % Step 4: Find pivot row (minimum positive ratio)
    ratios = b ./ A(:, pivot_col);
    ratios(ratios <= 0) = Inf; % Ignore negative or zero entries
    [min_ratio, pivot_row] = min(ratios);

    % Check for unboundedness
    if isinf(min_ratio)
        error("The problem is unbounded.");
    end
    disp("Pivot Row: " + pivot_row);

    % Step 5: Perform pivoting
    key_element = A(pivot_row, pivot_col);
    disp("Key Element: " + key_element);

    % Normalize the pivot row
    S(pivot_row, :) = S(pivot_row, :) / key_element;

    % Update the other rows
    for i = 1:m
        if i ~= pivot_row
            S(i, :) = S(i, :) - S(i, pivot_col) * S(pivot_row, :);
        end
    end
end

```

```

% Update Cb and b
Cb(pivot_row) = C(pivot_col);
b = S(:, end);

% Update A
A = S(:, 1:n);

% Display modified simplex table
disp("Modified Simplex Table:");
disp(S);
end

% Step 6: Display the final solution
disp("Final Simplex Tableau:");
disp(S);

% Initialize basic variable solutions as zeros
basic_solution = zeros(n, 1);

% Extract basic variables from tableau
for i = 1:m
    % Find the column with a single '1' in the current row
    for j = 1:n
        if A(i, j) == 1 && sum(A(:, j) == 1) == 1
            basic_solution(j) = b(i); % Assign value from RHS
            break;
        end
    end
end
end

% Display results
disp("Optimal Solution for Dual:");
disp(basic_solution);

% Calculate and display the optimal value
disp("Optimal Value for Dual:");
optimal_value = Cb' * b;
disp(optimal_value); % Negate for minimization problem

```

Question 1: A patient in a hospital is required to have at least 84 units of drug A and 120 units of drug B each day. Each gram of substance M contains 10 units of drug A and 8 units of drug B, and each gram of substance N contains 2 units of drug A and 4 units of drug B. Now suppose that both M and N contain an undesirable drug C, 3 units per gram in M and 1 unit per gram in N. How many grams of substances M and N should be mixed to meet the minimum daily requirements at the same time minimize the intake of drug C? How many units of the undesirable drug C will be in this mixture?

Input - > $A = \begin{bmatrix} 10 & 8 & 1 & 0; \\ 2 & 4 & 0 & 1 \end{bmatrix}$; $b = \begin{bmatrix} 3; 1 \end{bmatrix}$; $C = \begin{bmatrix} 84 & 120 & 0 & 0 \end{bmatrix}$;

Output ->

Zj-Cj:

-84 -120 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 2

Key Element: 4

Modified Simplex Table:

Columns 1 through 4

6	0	1	-2
1/2	1	0	1/4

Column 5

1
1/4

Zj-Cj:

-24 0 0 30

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 1

Key Element: 6

Modified Simplex Table:

Columns 1 through 4

1	0	1/6	-1/3
0	1	-1/12	5/12

Column 5

1/6
1/6

Zj-Cj:

0 0 4 22

Optimal solution found.

Final Simplex Tableau:

Columns 1 through 4

1	0	1/6	-1/3
0	1	-1/12	5/12

Column 5

1/6
1/6

Optimal Solution for Dual:

1/6
1/6
0
0

Optimal Value for Dual:

34

Question 2: A company manufactures two products, X and Y by using three machines A, B, and C. Machine A has 4 hours of capacity available during the coming week. Similarly, the available capacity of machines B and C during the coming week is 24 hours and 35 hours respectively. One unit of product X requires one hour of Machine A, 3 hours of machine B and 10 hours of machine C. Similarly one unit of product Y requires 1 hour, 8 hour and 7 hours of machine A, B and C respectively. When one unit of X is sold in the market, it yields a profit of \$5 per product and that of Y is \$7 per unit. Formulate the linear programming problem to maximize the profit by using graphical method. Also write the dual of the same

Input -> $A = \begin{bmatrix} 1 & 3 & 10 & -1 & 0 & 1 & 0 \\ 1 & 8 & 7 & 0 & -1 & 0 & 1 \end{bmatrix}$ $b = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$ $C = \begin{bmatrix} -4 & -24 & -35 & 0 & 0 & -M & -M \end{bmatrix}$
 $C_b = \begin{bmatrix} -M \\ -M \end{bmatrix}$;

Output ->

Zj-Cj:

Columns 1 through 4

-19996 -109976 -169965 10000

Columns 5 through 7

10000 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 3

Pivot Row: 1

Key Element: 10

Modified Simplex Table:

Columns 1 through 4

1/10 3/10 1 -1/10
3/10 59/10 0 7/10

Columns 5 through 8

0 1/10 0 1/2
-1 -7/10 1 7/2

Zj-Cj:

Columns 1 through 4

-5999/2 -117973/2 0 -13993/2

Columns 5 through 7

10000 33993/2 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 2

Key Element: 5.9

Modified Simplex Table:

Columns 1 through 4

5/59 0 1 -8/59
3/59 1 0 7/59

Columns 5 through 8

3/59 8/59 -3/59 19/59
-10/59 -7/59 10/59 35/59

Zj-Cj:

Columns 1 through 4

-11/59 0 0 112/59

Columns 5 through 7

135/59 99981/10 69984/7

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 1

Key Element: 0.084746

Modified Simplex Table:

Columns 1 through 4

1 0 59/5 -8/5
0 1 -3/5 1/5

Columns 5 through 8

3/5 8/5 -3/5 19/5
-1/5 -1/5 1/5 2/5

Zj-Cj:

Columns 1 through 4

0 0 11/5 8/5

Columns 5 through 7

12/5 49992/5 49988/5

Optimal solution found.

Final Simplex Tableau:

Columns 1 through 4

1 0 59/5 -8/5
0 1 -3/5 1/5

Columns 5 through 8

3/5 8/5 -3/5 19/5
-1/5 -1/5 1/5 2/5

Optimal Solution:

19/5
2/5
0
0
0
0
0
0

Optimal Value:

124/5

Question 3: In this problem, a patient visits the doctor to get treatment for ill health. The doctor examines the patient and advises him to consume at least 40 units of vitamin A and 50 units of vitamin B daily for a specified time period. He also advises the patient that to get vitamin A and vitamin B he has to drink tonic X and tonic Y that have both vitamin A and vitamin B in a proportion. One unit of tonic X consists 2 units of vitamin A and 3 units of vitamin B and one unit of tonic Y consists of 4 units of vitamin A and 2 units of vitamin B. These tonics are available in medical shops at a cost of Rs.3.00 and Rs.2.50 per unit of X and Y respectively. Now the problem of patient is how much of X and how much of Y is to be purchased from the shop to minimise the total cost and at the same time he can get required amounts of vitamins A and B.

Input - $A = [3 \ 5 \ 1 \ 0; 4 \ 2 \ 0 \ 1]$ $b = [60; 80]$ $C = [8 \ 11 \ 0 \ 0]$

Output ->

Zj-Cj:

-8 -11 0 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 2

Pivot Row: 1

Key Element: 5

Modified Simplex Table:

Columns 1 through 4

3/5	1	1/5	0
14/5	0	-2/5	1

Column 5

12
56

Zj-Cj:

-7/5 0 11/5 0

Solution is not optimal, proceeding with iteration.

Pivot Column: 1

Pivot Row: 1

Key Element: 0.6

Modified Simplex Table:

Columns 1 through 4

1	5/3	1/3	0
0	-14/3	-4/3	1

Column 5

20
0

Zj-Cj:

0 7/3 8/3 0

Optimal solution found.

Final Simplex Tableau:

Columns 1 through 4

1	5/3	1/3	0
0	-14/3	-4/3	1

Column 5

20

0

Optimal Solution:

20

0

0

0

Optimal Value:

160

Dual- Simplex Method

Date: 14 Feb 2025

Code

```
clear all;
clc;
format 'rational';

% Inputs A b C
% Maximize  $Z = 5x + 6y$ 
% Subject to:
%  $2x + 4y \leq 16$ 
%  $3x + 2y \leq 12$ 
%  $x, y \geq 0$ 

% Define coefficient matrix A and RHS b
A = [2 4; 3 2];
b = [16; 12];
C = [5 6]; % For maximization, no need to negate

% Convert to Standard Simplex Form (add slack variables)
[m, n] = size(A);
A = [A eye(m)]; % Add slack variables
C = [C zeros(1, m)]; % Extend C with zeros for slack variables
Cb = zeros(m, 1); % Initial basis coefficients for slack variables

% Initialize tableau
S = [A b];
Basic = n+1:n+m; % Keep track of basic variables

% Main loop
disp("Initial Simplex Table:");
disp(S);

while true
    % Calculate  $Z_j - C_j$ 
    Zj = Cb' * A;
    relative_cost = Zj - C;
    disp("Zj-Cj:");
    disp(relative_cost);

    % Check for optimality
    if all(relative_cost <= 0)
        break; % Optimal solution found
    end

    % Find pivot column (most positive  $Z_j - C_j$ )
```

```

[~, pivot_col] = max(relative_cost);

% Find pivot row using minimum ratio test
ratios = b ./ A(:, pivot_col);
ratios(A(:, pivot_col) <= 0) = Inf;
disp("ratios:");
disp(ratios);
[min_ratio, pivot_row] = min(ratios);

if min_ratio == Inf
    error('Problem is unbounded');
end

% Pivot
pivot_element = A(pivot_row, pivot_col);

% Update the tableau
S(pivot_row, :) = S(pivot_row, :) / pivot_element;
for i = 1:m
    if i ~= pivot_row
        S(i, :) = S(i, :) - S(i, pivot_col) * S(pivot_row, :);
    end
end

% Update basis
Basic(pivot_row) = pivot_col;
Cb(pivot_row) = C(pivot_col);

% Update A and b
A = S(:, 1:end-1);
b = S(:, end);

disp("Modified Simplex Table:");
disp(S);
end

% Calculate final solution
x = zeros(n, 1);
for i = 1:m
    if Basic(i) <= n
        x(Basic(i)) = b(i);
    end
end

% Calculate optimal value
optimal_value = C(1:n) * x;

% Display results
disp('Optimal Solution:');
disp(x);

```

```
disp('Optimal Value:');
disp(optimal_value);
```

Q1. An animal food company at least produce 200kg of ingredients x1 and x2 daily. x1 costs Rs.3 per kg and x2 costs Rs.8 per kg . Not more than 80 kg of x1 can be used and at least 60kg of x2 must be used. Formulate the LP model to minimize the cost and compute the optimal cost .

Input -> $A = [-1 \ -1; \ 1 \ 0; \ 0 \ -1]$; $b = [-200; 80; -60]$; $C = [3 \ 8]$;

Output ->

Initial Simplex Table :

-1	-1	1	0	0	-200
1	0	0	1	0	80
0	-1	0	0	1	-60

Zj-Cj :

-3	-8	0	0	0
----	----	---	---	---

Modified Simplex Table :

1	1	-1	0	0	200
0	-1	1	1	0	-120
0	-1	0	0	1	-60

Zj-Cj :

0	-5	-3	0	0
---	----	----	---	---

Modified Simplex Table :

1	0	0	1	0	80
0	1	-1	-1	0	120
0	0	-1	-1	1	60

Optimal Solution:

80
120

Optimal Value:

1200

Q2. Egg contains 6 units A and 7 units of B per gram and costs 12 paise per gram. Milk contains 8 units of vitamin A and 12 units of Vitamin B and costs 20 paise per gram. The daily minimum requirement of vitamin A and vitamin B are 100 units and 120 units respectively. Find the optimal product mix.

Input -> $A = [-6 \ -8; \ -7 \ -12]$; $b = [-100; -120]$; $C = [12 \ 20]$

Output ->

Initial Simplex Table :
Columns 1 through 4

-6	-8	1	0
-7	-12	0	1

Column 5

-100
-120

Zj-Cj :
-12 -20 0 0

ratios:
12/7 5/3 1/0 1/0

Modified Simplex Table :
Columns 1 through 4

-4/3	0	1	-2/3
7/12	1	0	-1/12

Column 5

-20
10

Zj-Cj :
-1/3 0 0 -5/3

ratios:
1/4 1/0 1/0 5/2

Modified Simplex Table :
Columns 1 through 4

1	0	-3/4	1/2
0	1	7/16	-3/8

Column 5

15
5/4

Optimal Solution:
15
5/4

Optimal Value:

Q3. A factory produces two types of toys, A and B. Toy A requires 2 hours of machine time and 3 hours of labor. Toy B requires 4 hours of machine time and 2 hours of labor. The factory has a maximum of 16 hours of machine time and 12 hours of labor available. The profit for each toy A is \$5, and the profit for each toy B is \$6. Determine the optimal production mix (i.e., how many of each toy to produce) to maximize the total profit.

Input -> $A = \begin{bmatrix} 2 & 4 \\ 3 & 2 \end{bmatrix}$ $b = \begin{bmatrix} 16 \\ 12 \end{bmatrix}$ $C = \begin{bmatrix} 5 & 6 \end{bmatrix}$

Output ->

Initial Simplex Table:

Columns 1 through 4

2	4	1	0
3	2	0	1

Column 5

16
12

Zj-Cj:

-5	-6	0	0
----	----	---	---

Optimal Solution:

0
0

Optimal Value:

0

DAY-4

Transportation Problem (North West Corner Method with MODI)

Date: 21st Feb 2025

Code

```
clc; clear; close all;

%% Input -> supply demand and cost
supply = [11 13 19]; % Supply for each source
demand = [6 10 12 15]; % Demand for each destination

% Cost matrix
cost = [21 16 25 43;
        25 18 14 53;
        32 27 15 41];

%% Balance the problem if necessary
total_supply = sum(supply);
total_demand = sum(demand);
if total_supply == total_demand
    fprintf('Balanced Transportation Problem.\n');
else
    if total_supply > total_demand
        demand = [demand, total_supply - total_demand];
        cost = [cost, zeros(size(cost,1), 1)]; % Zero cost for dummy destination
        fprintf('Unbalanced: Dummy column added.\n');
    else
        supply = [supply, total_demand - total_supply];
        cost = [cost; zeros(1, size(cost,2))]; % Zero cost for dummy source
        fprintf('Unbalanced: Dummy row added.\n');
    end
end

[m, n] = size(cost);
allocation = zeros(m, n);

%% Compute Initial Basic Feasible Solution (North-West Corner)
supply_left = supply;
demand_left = demand;
i = 1; j = 1;
while i <= m && j <= n
    alloc = min(supply_left(i), demand_left(j));
    allocation(i, j) = alloc;
    supply_left(i) = supply_left(i) - alloc;
    demand_left(j) = demand_left(j) - alloc;
    if supply_left(i) == 0 && i < m
        i = i + 1;
    elseif demand_left(j) == 0 && j < n
        j = j + 1;
    elseif supply_left(i)==0 && demand_left(j)==0 && i < m && j < n
        i = i + 1;
        j = j + 1;
    else
        break;
    end
end
```

```

disp('Initial Basic Feasible Solution (NW Corner):');
disp(allocation);

%% MODI Method to Optimize the Solution
while true
    % Step 1: Compute u and v potentials for allocated cells.
    u = NaN(m,1); v = NaN(1,n);
    u(1) = 0; % Arbitrary assignment
    updated = true;
    while updated
        updated = false;
        for i = 1:m
            for j = 1:n
                if allocation(i,j) > 0
                    if ~isnan(u(i)) && isnan(v(j))
                        v(j) = cost(i,j) - u(i);
                        updated = true;
                    elseif isnan(u(i)) && ~isnan(v(j))
                        u(i) = cost(i,j) - v(j);
                        updated = true;
                    end
                end
            end
        end
    end

    % Step 2: Compute the opportunity cost (delta) for each unallocated cell.
    delta = NaN(m,n);
    for i = 1:m
        for j = 1:n
            if allocation(i,j) == 0
                delta(i,j) = cost(i,j) - (u(i) + v(j));
            end
        end
    end

    % Check for optimality (if all deltas are nonnegative, we are done)
    if all(delta(~isnan(delta)) >= 0)
        fprintf('Optimal Solution Found:\n');
        disp(allocation);
        total_cost = sum(sum(allocation .* cost));
        fprintf('Minimum Cost: %d\n', total_cost);
        break;
    end

    % Step 3: Select the entering cell (most negative delta)
    [minDelta, linearIndex] = min(delta(:));
    [enter_i, enter_j] = ind2sub(size(delta), linearIndex);
    fprintf('Improving solution using cell (%d, %d) with delta = %d\n', enter_i,
    enter_j, minDelta);

    % Step 4: Find a closed loop (cycle) including the entering cell.
    cycle = findCycle(allocation, [enter_i, enter_j]);
    if isempty(cycle)
        error('No cycle found. Check the feasibility or degeneracy handling.');
```

% The cycle is returned as a list of coordinates (first equals last).

```

% Assign alternating + and - signs along the cycle.
% The entering cell (first cell) gets a '+' sign.
numPoints = size(cycle,1) - 1; % last is duplicate of first
signs = repmat([1; -1], ceil(numPoints/2), 1);
signs = signs(1:numPoints);

% Step 5: Determine theta = minimum allocation on cells with negative sign.
theta = inf;
for k = 2:2:numPoints
    i_idx = cycle(k,1);
    j_idx = cycle(k,2);
    if allocation(i_idx, j_idx) < theta
        theta = allocation(i_idx, j_idx);
    end
end

% Step 6: Update allocations along the cycle.
for k = 1:numPoints
    i_idx = cycle(k,1);
    j_idx = cycle(k,2);
    if mod(k,2)==1 % Positive positions
        allocation(i_idx, j_idx) = allocation(i_idx, j_idx) + theta;
    else % Negative positions
        allocation(i_idx, j_idx) = allocation(i_idx, j_idx) - theta;
    end
end
end

%% --- Nested Functions for Cycle Detection ---
% The following functions recursively search for a closed path (cycle)
% that starts and ends at the entering cell. Moves alternate between
% horizontal and vertical directions.

function cycle = findCycle(allocation, start)
    % Initiate a recursive search for a cycle starting at 'start'.
    path = start; % current path (each row: [row, col])
    cycle = searchCycle(allocation, start, path, 0);
end

function foundCycle = searchCycle(allocation, start, currentPath, lastDir)
    % lastDir: 0 = undefined, 1 = horizontal, 2 = vertical.
    current = currentPath(end,:);
    % If cycle is complete (at least 4 points) and returned to start, return path.
    if size(currentPath,1) >= 4 && isequal(current, start)
        foundCycle = currentPath;
        return;
    end
    foundCycle = [];
    [rows, cols] = size(allocation);
    moves = [];

    % Determine allowed moves based on last move's direction.
    if lastDir == 0 || lastDir == 2
        % Allowed to move horizontally (same row).
        r = current(1);
        for j = 1:cols
            if j ~= current(2)
                if (allocation(r,j) > 0) || (isequal([r, j], start))
                    moves = [moves; r, j, 1]; % 1 indicates horizontal move
                end
            end
        end
    else
        % Allowed to move vertically (same column).
        c = current(2);
        for r = 1:rows
            if r ~= current(1)
                if (allocation(r,c) > 0) || (isequal([r, c], start))
                    moves = [moves; r, c, 2]; % 2 indicates vertical move
                end
            end
        end
    end

    % Recursively search for a cycle.
    foundCycle = searchCycle(allocation, start, [currentPath; moves], lastDir);
end

```

```

        end
    end
end
if lastDir == 0 || lastDir == 1
    % Allowed to move vertically (same column).
    c = current(2);
    for i = 1:rows
        if i ~= current(1)
            if (allocation(i,c) > 0) || (isequal([i, c], start))
                moves = [moves; i, c, 2]; % 2 indicates vertical move
            end
        end
    end
end

% Explore each possible move.
for k = 1:size(moves,1)
    nextCell = moves(k,1:2);
    moveDir = moves(k,3);
    % Avoid immediate backtracking.
    if size(currentPath,1) > 1 && isequal(nextCell, currentPath(end-1,:))
        continue;
    end
    % Avoid repeating cells (except if it's the starting cell to complete the
cycle)
    if any(ismember(currentPath(1:end-1,:), nextCell, 'rows'))
        if isequal(nextCell, start) && size(currentPath,1) >= 3
            foundCycle = [currentPath; nextCell];
            return;
        else
            continue;
        end
    end
    newPath = [currentPath; nextCell];
    foundCycle = searchCycle(allocation, start, newPath, moveDir);
    if ~isempty(foundCycle)
        return;
    end
end
end
end

```

Question 1 : A company has three warehouses (sources) with supplies of 11, 13, and 19 units, respectively. These units need to be shipped to four retail stores (destinations) with demands of 6, 10, 12, and 15 units, respectively. The cost of shipping one unit from each warehouse to each retail store is given by the following cost matrix:
[21 16 25 43; 25 18 14 53; 32 27 18 41]

Input -> supply = [11 13 19]; demand = [6 10 12 15];
cost = [21 16 25 43; 25 18 14 53; 32 27 18 41];

Output ->

Balanced Transportation Problem.

Initial Basic Feasible Solution (NW Corner):

6	5	0	0
0	5	8	0

0 0 4 15

Optimal Solution Found:

6 5 0 0
0 5 8 0
0 0 4 15

Minimum Cost: 1095

Question 2 : A distribution company has three factories (sources) with available inventory of 15, 20, and 10 units, respectively. These units need to be transported to four retail outlets (destinations) with demands of 8, 12, 10, and 15 units, respectively. The cost of transporting one unit from each factory to each retail outlet is given by the following cost matrix:
[10 12 15 18; 14 16 20 22; 18 20 25 30]

Input -> supply = [15 20 10]; demand = [8 12 10 15];
cost = [10 12 15 18; 14 16 20 22; 18 20 25 30];

Output ->

Balanced Transportation Problem.

Initial Basic Feasible Solution (NW Corner):

8 7 0 0
0 5 10 5
0 0 0 10

Improving solution using cell (3, 1) with delta = -4

Improving solution using cell (1, 3) with delta = -5

Improving solution using cell (3, 2) with delta = -5

Improving solution using cell (2, 1) with delta = -1

Optimal Solution Found:

0 5 10 0
5 0 0 15
3 7 0 0

Minimum Cost: 804

Question 3 : A manufacturing firm has three production plants (sources) with capacities of 25, 30, and 20 units, respectively. These units must be shipped to four distribution centers (destinations) with requirements of 10, 15, 25, and 25 units, respectively. The cost of shipping one unit from each production plant to each distribution center is given by the following cost matrix:

[5 8 10 12; 6 9 11 13; 7 10 12 14]

Input -> supply = [25 30 20]; demand = [10 15 25 25];
cost = [5 8 10 12; 6 9 11 13; 7 10 12 14];

Output ->

Balanced Transportation Problem.

Initial Basic Feasible Solution (NW Corner):

10 15 0 0

0	0	25	5
0	0	0	20

Optimal Solution Found:

10	15	0	0
0	0	25	5
0	0	0	20

Minimum Cost: 790

DAY-4

Assignment Problem

Date: 28th Feb 2025

Code

```
% Hungarian Algorithm
clc; clear;

%% Example Cost Matrix (minimization problem)
% Input -> costMatrix
costMatrix = [10 3 3 2 8;
              9 7 8 2 7;
              7 5 6 2 4;
              3 5 8 2 4;
              9 10 9 6 10];

%% Step 0: Check if the problem is balanced (square matrix)
[m, n] = size(costMatrix);
if m ~= n
    fprintf('The problem is unbalanced. Balancing the matrix by adding dummy rows/columns.\n');
    if m < n
        costMatrix = [costMatrix; zeros(n - m, n)];
    else
        costMatrix = [costMatrix, zeros(m, m - n)];
    end
    [m, n] = size(costMatrix);
end

fprintf('Balanced Cost Matrix:\n');
disp(costMatrix);

%% Step 1: Row Reduction
rowMin = min(costMatrix, [], 2);
rowRed = costMatrix - repmat(rowMin, 1, n);
fprintf('Row Reduced Matrix:\n');
disp(rowRed);

%% Step 2: Column Reduction
colMin = min(rowRed, [], 1);
colRed = rowRed - repmat(colMin, m, 1);
fprintf('Column Reduced Matrix:\n');
disp(colRed);

%% Step 3: Initial Zero Marking (Starring)
mask = zeros(n); % 0: no mark, 1: starred zero, 2: primed zero.
rowCover = zeros(n,1);
colCover = zeros(n,1);

for i = 1:n
    for j = 1:n
        if colRed(i,j) == 0 && ~rowCover(i) && ~colCover(j)
            mask(i,j) = 1; % Star the zero.
            rowCover(i) = 1; % Mark row as having a starred zero.
            colCover(j) = 1; % Mark column as having a starred zero.
        end
    end
end
```

```

    end
end

% Clear covers for the next steps.
rowCover(:) = 0;
colCover(:) = 0;

% Cover each column containing a starred zero.
for j = 1:n
    if any(mask(:, j) == 1)
        colCover(j) = 1;
    end
end

%% Step 4: Main Loop of the Algorithm
while sum(colCover) < n
    % Find an uncovered zero.
    uncovered = (colRed == 0) & (repmat(rowCover, 1, n) == 0) & (repmat(colCover',
n, 1) == 0);
    [r, c] = find(uncovered, 1);

    while isempty(r)
        % Adjust the matrix if no uncovered zero is found.
        minVal = min(min(colRed(rowCover==0, colCover==0)));
        colRed(rowCover==1, :) = colRed(rowCover==1, :) + minVal;
        colRed(:, colCover==0) = colRed(:, colCover==0) - minVal;
        uncovered = (colRed == 0) & (repmat(rowCover, 1, n) == 0) &
(repmat(colCover', n, 1) == 0);
        [r, c] = find(uncovered, 1);
    end

    % Prime the uncovered zero.
    mask(r, c) = 2;

    % Check for a starred zero in the same row.
    starCol = find(mask(r, :) == 1, 1);
    if ~isempty(starCol)
        rowCover(r) = 1;
        colCover(starCol) = 0;
    else
        % Construct an augmenting path.
        path = [r, c];
        done = false;
        while ~done
            last_col = path(end, 2);
            r_index = find(mask(:, last_col) == 1, 1);
            if isempty(r_index)
                done = true;
            else
                path = [path; r_index, last_col]; %#ok<AGROW>
                c_index = find(mask(path(end,1), :) == 2, 1);
                path = [path; path(end,1), c_index]; %#ok<AGROW>
            end
        end

        % Augment the path: star primed zeros and unstar starred zeros.
        for i = 1:size(path, 1)
            if mask(path(i,1), path(i,2)) == 1
                mask(path(i,1), path(i,2)) = 0;
            end
        end
    end
end

```

```

        else
            mask(path(i,1), path(i,2)) = 1;
        end
    end

    % Reset covers and erase all primes.
    rowCover(:) = 0;
    colCover(:) = 0;
    mask(mask == 2) = 0;

    % Cover columns with starred zeros.
    for j = 1:n
        if any(mask(:, j) == 1)
            colCover(j) = 1;
        end
    end
end

end

%% Step 5: Final Assignment
assignment = zeros(n,1);
for i = 1:n
    j = find(mask(i, :) == 1, 1);
    if ~isempty(j)
        assignment(i) = j;
    else
        assignment(i) = 0;
    end
end

fprintf('Final Assignment (row -> column):\n');
disp(assignment);

% Create and display the Assignment Matrix.
assignmentMatrix = zeros(n);
for i = 1:n
    if assignment(i) > 0
        assignmentMatrix(i, assignment(i)) = 1;
    end
end

fprintf('Assignment Matrix (1 indicates the chosen assignment):\n');
disp(assignmentMatrix);

% Compute the final cost using the original cost matrix.
finalCost = 0;
for i = 1:n
    finalCost = finalCost + costMatrix(i, assignment(i));
end
fprintf('Final Cost: %d\n', finalCost);

```

Question 1 : Cost Matrix = [21 16 25 43; 25 18 14 53; 32 27 18 41]

Input -> costMatrix = [21 16 25 43; 25 18 14 53; 32 27 18 41];

Output ->

The problem is unbalanced. Balancing the matrix by adding dummy rows/columns.

Balanced Cost Matrix:

21	16	25	43
25	18	14	53
32	27	18	41
0	0	0	0

Row Reduced Matrix:

5	0	9	27
11	4	0	39
14	9	0	23
0	0	0	0

Column Reduced Matrix:

5	0	9	27
11	4	0	39
14	9	0	23
0	0	0	0

Final Assignment (row -> column):

1
2
3
4

Assignment Matrix (1 indicates the chosen assignment):

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Final Cost: 57

Question 2 : Cost Matrix =[10 12 15 18; 14 16 20 22; 18 20 25 30]

Input - > costMatrix = [10 12 15 18; 14 16 20 22; 18 20 25 30];

Output ->

The problem is unbalanced. Balancing the matrix by adding dummy rows/columns.

Balanced Cost Matrix:

10	12	15	18
14	16	20	22
18	20	25	30
0	0	0	0

Row Reduced Matrix:

0	2	5	8
0	2	6	8
0	2	7	12
0	0	0	0

Column Reduced Matrix:

0	2	5	8
0	2	6	8
0	2	7	12
0	0	0	0

Final Assignment (row -> column):

3
2
1
4

Assignment Matrix (1 indicates the chosen assignment):

0	0	1	0
0	1	0	0
1	0	0	0
0	0	0	1

Final Cost: 49

Question 3 : Cost Matrix = [5 8 10 12; 6 9 11 13; 7 10 12 14]

Input -> costMatrix = [5 8 10 12; 6 9 11 13; 7 10 12 14];

Output ->

The problem is unbalanced. Balancing the matrix by adding dummy rows/columns.

Balanced Cost Matrix:

5	8	10	12
6	9	11	13
7	10	12	14
0	0	0	0

Row Reduced Matrix:

0	3	5	7
0	3	5	7
0	3	5	7
0	0	0	0

Column Reduced Matrix:

0	3	5	7
0	3	5	7
0	3	5	7
0	0	0	0

Final Assignment (row -> column):

3
2
1
4

Assignment Matrix (1 indicates the chosen assignment):

0	0	1	0
---	---	---	---

0	1	0	0
1	0	0	0
0	0	0	1

Final Cost: 26

DAY-5

Region Elimination Method (Fibonacci Search)

Date: 21st March 2025

Code 1 (Minimize)

```
clc;
clear all;

% Function to minimize
f = @(x) x.^2;

% Interval [a, b]
a = -5;
b = 15;

% Number of iterations
n = 7;

% Generate Fibonacci numbers
F = [1, 1];
for i = 3:n+2
    F(i) = F(i-1) + F(i-2);
end

% Fibonacci Search
for k = 1:n
    L = (F(n-k+1) / F(n+2)) * (b - a);
    x1 = a + L;
    x2 = b - L;

    f1 = f(x1);
    f2 = f(x2);

    if f1 < f2
        b = x2; % Move left
    else
        a = x1; % Move right
    end
end

% Final optimal estimate
x_opt = (a + b) / 2;
f_opt = f(x_opt);

fprintf('\nMinimum at x=%.4f, f(x)=%.4f\n', x_opt, f_opt);
```

Code 2 (Maximize)

```
clc;
clear all;

% Function to maximize
```

```

f = @(x) x.^2;

% Interval [a, b]
a = -5;
b = 15;

% Number of iterations
n = 7;

% Generate Fibonacci numbers
F = [1, 1];
for i = 3:n+2
    F(i) = F(i-1) + F(i-2);
end

% Fibonacci Search
for k = 1:n
    L = (F(n-k+1) / F(n+2)) * (b - a);
    x1 = a + L;
    x2 = b - L;

    f1 = f(x1);
    f2 = f(x2);

    if f1 > f2
        b = x2; % Move left
    else
        a = x1; % Move right
    end
end

% Final optimal estimate
x_opt = (a + b) / 2;
f_opt = f(x_opt);

fprintf('\nMaximum at x=%.4f, f(x)=%.4f\n', x_opt, f_opt);

```

Question 1: Min $f(x) = x^2 + 54/x$ | $a=0.1$ | $b=5$

Input-> $f = x.^2 + 54./x$ $a=0.1$ $b=5$

Output-> Minimum at $x=-1.7433$, $f(x)=-27.9366$

Question 2: Max $f(x) = x^2 + 54/x$ | $a=0.1$ | $b=5$

Input-> $f = x.^2 + 54./x$ $a=0.1$ $b=5$

Output-> Maximum at $x=11.7433$, $f(x)=142.5035$

Question 1: Max $f(x) = 10 + x^3 - 2x$ | $a=-5$ | $b=5$

Input-> $f = 10 + x.^3 - 2*x$ $a=-5$ $b=5$

Output-> Maximum at $x=11.7433$, $f(x)=1605.9742$

Newton - Raphson Method

Date: 21st March 2025

Code

```
clc; clear;

syms x;
f = 2*sin(x)-(x^2)/10; % Input Function

% Compute first and second derivatives
f1 = diff(f, x); % First derivative
f2 = diff(f1, x); % Second derivative

% Convert symbolic to function
f1_func = matlabFunction(f1);
f2_func = matlabFunction(f2);

% Initial guess
x_k = 0;
tol = 1e-6;

for i = 1:100
    x_new = x_k - f1_func(x_k) / f2_func(x_k); % Newton's update

    if abs(x_new - x_k) < tol
        break;
    end
    x_k = x_new;
end

fprintf('Optimal x = %.6f, f(x) = %.6f\n', x_k, subs(f, x_k));
```

Question 1: Use Newton's method to find the maximum of $f(x) = 2 \sin x - x^2/10$ with an initial guess of $x_0 = 2.5$.

Input-> $f = 2 \sin(x) - (x^2)/10$;

Output-> Optimal $x = 1.427552$, $f(x) = 1.775726$

Question 2: Use Newton's method to find the maximum of $f(x) = \log(x) - x^2/4$ with an initial guess of $x_0 = 2.0$.

Input-> $f = \log(x) - (x^2)/4$

Output-> Optimal $x = 1.414214$, $f(x) = -0.153426$

Question 1: Use Newton's method to find the maximum of $f(x) = e^{-x} - x^2/5$ with an initial guess of $x_0 = 0.5$.

Input-> $f = \exp(-x) - (x^2)/5$

Output-> Optimal $x = -2.455226$, $f(x) = 10.443443$